

Assessment Problem Statements

Python for GenAI & Data Analysis | App Development with GenAI

The two problem statements below each integrate topics from both tracks into a single, realistic mini-task. They go a step beyond isolated syntax exercises by requiring learners to combine object-oriented design, error handling, and core GenAI concepts within one small project.

Each can be completed individually and is scoped for a 1.5-2 hour assessment window.

Problem Statement 1: Mini RAG-Ready Document Processor

Primary topics combined: OOP & custom exceptions, logging, text preprocessing & chunking, keyword-based retrieval, simulated embeddings, LLM context-window concepts

Difficulty: Slightly above basic — requires combining 3–4 sub-topics into one working pipeline

Scenario

Your team is prototyping a lightweight document Q&A tool before committing to a full vector-database solution. You have been asked to build the preprocessing and retrieval layer that will later feed a large language model (LLM). Because the LLM has a limited context window, documents must be split into well-sized chunks, cleaned, and indexed so the most relevant chunks can be retrieved for any given query.

Requirements

1. Design a `DocumentStore` class (OOP) that loads one or more plain-text documents and stores them internally.
2. Implement text preprocessing: lowercasing, punctuation removal, and stopwords filtering using `Pandas/NumPy` or standard `Python`.
3. Implement a `chunk_document()` method that splits cleaned text into overlapping chunks of a configurable size (e.g., 200 words with a 20-word overlap), and briefly justify your chunk size in a code comment by relating it to an LLM's context-window limit.
4. Implement a keyword-based `retrieve(query, top_k)` method that returns the top-k most relevant chunks using a simple scoring method (e.g., term-frequency overlap or a basic TF-IDF-style score) — no external ML libraries required.
5. Attach a `simulate_embedding(chunk)` method that returns a fixed-length numeric vector (e.g., a hash-based or random-seeded `NumPy` array) to represent how real embeddings would later be plugged in for a RAG pipeline.
6. Define at least one custom exception (e.g., `EmptyDocumentError` or `ChunkSizeError`) and raise it under an invalid input condition; handle it gracefully with `try/except` at the call site.
7. Add logging (not print statements) at `INFO` level for major steps and at `ERROR` level when an exception is caught.
8. Write at least 3 unit tests (`unittest` or `pytest`) covering: a normal case, an empty-document edge case, and a retrieval query with no matches.

Deliverables

- A single `Python` module (or small package) with the `DocumentStore` class and helper functions.
- A short driver script demonstrating: `load` → `preprocess` → `chunk` → `simulate embeddings` → `retrieve` for a sample query.

- The unit test file and a short README (5–10 lines) noting any assumptions made.

Evaluation Criteria

- Correctness of chunking and retrieval logic (30%)
 - Sound OOP structure and use of custom exceptions (25%)
 - Logging, code quality, and readability (20%)
 - Test coverage of edge cases (15%)
 - Clarity of the written justification linking chunk size to context-window limits (10%)
-

Problem Statement 2: Conversational AI Concept Advisor

Primary topics combined: AI vs ML vs DL vs GenAI, LLM concepts (tokens/context window), OpenAI/ChatGPT API, prompt engineering, OOP wrapper design, error handling & logging

Difficulty: Slightly above basic — requires combining a conceptual layer with a small working API integration

Scenario

A training team wants a small internal tool that answers learners' questions about AI terminology (e.g., “What is the difference between ML and DL?”) by calling an LLM through the OpenAI/ChatGPT API, while enforcing a consistent, well-structured response style through prompt engineering. You have been asked to build the wrapper class that manages this behind the scenes.

Requirements

1. Design an `AIConceptAdvisor` class (OOP) that wraps calls to the OpenAI/ChatGPT API (a mock/stub function is acceptable if no API key is available; clearly mark it as such).
2. Implement a system prompt (prompt engineering) that instructs the model to: (a) classify the question as being about AI, ML, DL, or GenAI, and (b) answer in under 100 words at a beginner level.
3. Implement a method `estimate_tokens(text)` that gives an approximate token count for a prompt (e.g., using a simple word-based or character-based heuristic), and use it to warn the user — via logging — if a request would likely exceed a configurable context-window limit (e.g., 2048 tokens).
4. Add a short conceptual explanation method or docstring in the class that briefly distinguishes AI, ML, DL, and GenAI, and references how a neural network (ANN) underlies DL, so the class can optionally surface this as a fallback answer if the API call fails.
5. Define and raise a custom exception (e.g., `TokenLimitExceededError` or `APICallError`) for the context-window-exceeded case and for a simulated API failure; handle both with `try/except` and a graceful fallback message.
6. Add logging for each request (prompt sent, token estimate, success/failure) instead of print statements.
7. Write at least 2 unit tests: one verifying correct question classification/formatting of the prompt, and one verifying that `TokenLimitExceededError` is raised and handled when the input is deliberately oversized.

Deliverables

- A single Python module with the `AIConceptAdvisor` class.
- A short driver script showing three sample questions being asked and answered (or gracefully failing).
- The unit test file and a short README noting whether a real or mocked API call was used.

Evaluation Criteria

- Correct, well-structured prompt design and token-estimate logic (30%)
 - Sound OOP structure and use of custom exceptions (25%)
 - Logging, code quality, and readability (20%)
 - Test coverage of the classification and token-limit cases (15%)
 - Accuracy and clarity of the AI/ML/DL/GenAI conceptual explanation (10%)
-

Note: Both problem statements are designed to be completed without paid API access — a mocked or stubbed API response is acceptable wherever the OpenAI/ChatGPT API is referenced, as long as the surrounding logic (prompting, token estimation, error handling) is implemented for real.